Document 18 — Final Production Multi-Agent Capstone → Project 5

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

Everything. Docs 01-17.

How to Read & Practice This Document

- **What:** a full production multi-agent system, built from unclear business requirements — not a spec I've already solved for you.
- **Why:** this is the final piece of the whole curriculum — it's meant to be genuinely hard, and it's what you'll actually show in a portfolio or talk through in a final-round interview.
- **When:** now — this is where everything from Docs 01-17 gets used together, for real, all at once.
- **How to practice:** you'll be given requirements, not a design (see below). Design it in writing first, before any code. I review it like a senior architect at each stage — I won't correct you line-by-line unless you ask.

Jump to: What You'll Be Given · What You'll Design and Build · Practice Exercises

The Story — what this document is actually building

Every document before this one handed you something to build against — a spec, a set of test cases, a Build Task with a fixed shape. This one doesn't. You're handed the same thing a real engineer is handed on day one of a real job: a pile of business needs, constraints, and a scale target, with no one telling you what the architecture should look like. Turning that into a working system is the actual skill this whole curriculum has been building toward — everything from Doc01's config loader to Doc19's deploy gate exists so that, right now, none of it is new to you, only the combination is.

This is Project 5 — the same ContentForge system from Project 4, but taken from "working demo" to "something you'd actually trust with real users and real money." That means real requirements gathering, a design you commit to in writing before touching code, and then defending that design

under the same kind of pushback a senior architect would actually give you — because a design nobody has challenged yet is just a guess that hasn't been tested.

There's no fixed practice track here, on purpose — the capstone's five stages (design → build → test → deploy → defend) *are* the exercise. The three practice exercises below give you a fixed place to rehearse the "defend it" part specifically, since that's the part most people have never actually done before doing it for real.

What You'll Be Given (not designed for you)

- Business requirements
- Functional requirements
- Non-functional requirements (speed, cost limit, reliability targets)
- Constraints
- Expected scale / expected users
- Security requirements
- Cost limits

(Given to you live when you say START DOCUMENT 18 — kept out of this file on purpose, since part of the exercise is working from a live requirements handoff, not a checklist.)

What You'll Design and Build

- Agents (which pattern, following Doc11's reasoning)
- Tools
- Workflow (LangGraph)
- State (shared vs. private)
- RAG
- Database
- APIs (FastAPI, per Doc12)
- Login/permissions
- Testing (per Doc13)
- Watching/monitoring
- Deployment (Docker, per Doc12)

Go Deeper (Optional)

No new material at this stage — you should notice you don't need any. If you find yourself needing to re-read a Doc04-13 guide to keep going, that's a useful sign about which document needs more

practice before you continue here.

Practice Exercises

There's no separate practice track for the whole capstone — the capstone itself is the exercise. Treat each of its stages (design → build → test → deploy → defend) as one of the earlier documents' practice progressions, applied by yourself. The 3 exercises below give you a fixed, rehearsable version of the "defend it" stage specifically, the same design-critique style as Doc16 — reasoning about a design out loud, not writing code.

Where your notes live: `18_capstone/practice/` (`mkdir -p practice`). These are written critiques and defences, not code, so each one is a Markdown file. The capstone's own code lives in Project 5's folder.

```
practice/
├── capstone_review.md      Real-world
├── defend_tradeoff.md      Intermediate
└── design_self_review.md   Basic
```

Why each file exists:

- `capstone_review.md` — your notes after the cold review: which answers held, which were defaults, and which two answers turned out to contradict each other.
- `defend_tradeoff.md` — one debatable decision, the strongest objection you can build against it, and your honest answer — including the second round of pushback.
- `design_self_review.md` — the two-direction requirement check on your Stage-1 design, done before any code, so gaps and extra pieces are found on paper.

Jump to an exercise: Basic · Intermediate · Real-world

Basic — critique your own Stage-1 design before you build

- **What:** before writing any code, take your written capstone design and review it yourself as if you were the senior architect seeing it for the first time.
- **Why:** catching a weak design decision in writing costs you nothing; catching the same weak decision three weeks into building it costs you the three weeks.
- **When you'll hit this for real:** the moment you finish Stage 1 (design) and are tempted to jump straight into code — this exercise is that pause, made deliberate.
- **How to do it:** for each major piece of your design (agents, state, RAG, database), write one sentence naming the specific requirement it exists to satisfy. Anywhere you can't name one, flag it as either unnecessary complexity or a fix for a problem you don't have yet (see Doc16's "Requirements → architecture is matching, not inspiration").

- **Save as:** `practice/design_self_review.md`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — defend one non-obvious trade-off under pushback

- **What:** pick one genuinely debatable decision from your design, and defend it against the strongest reasonable objection you can think of.
- **Why:** a decision you can only defend against a weak objection isn't actually defended — this is where you find out if your reasoning holds up, before someone else finds the gap for you.
- **When you'll hit this for real:** any senior-level design review, and especially the "defend" stage of this capstone.
- **How to do it:** state the decision, state the strongest "why not X instead" you can construct against it, then answer it the way Doc16 describes — name the trade-off you actually weighed, what X would have cost you specifically, and be willing to say X is better if it genuinely is.
- **Save as:** `practice/defend_tradeoff.md`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — a cold design-review interview on your finished capstone

- **What:** let someone (me, or a peer) interview you about your finished capstone's design, cold, at senior-architect depth — not just "walk me through it" like Doc17's exercise, but real pushback on specific choices.
- **Why:** this is the actual final exam of the whole curriculum — everything from Doc16's design defense and Doc17's project walkthrough, aimed at the hardest, highest-stakes project you've built.
- **When you'll hit this for real:** a final-round interview, or any time you present this capstone as your portfolio's centerpiece.
- **How to do it:** say "interview me about my capstone" and answer as if the interviewer has full authority to challenge any decision — agents, state, RAG, deployment, cost — and expects you to hold your ground only where you actually should.
- **Save as:** `practice/capstone_review.md` (your notes right after the review).
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Expected Behavior / What "Done" Means

Meets every item in CURRICULUM.md §6, "Final capabilities" — done on your own, without AI-written code, and defendable when pushed on it.

Move On When

Never — this is the final piece. "Done" means it meets every item in §6 of CURRICULUM.md, and you can defend every design choice when pushed, without notes.

Say **START DOCUMENT 18** when Docs 01-17 are finished and you're ready for the requirements handoff.